

ELO329 - Diseño y Programación Orientados a Objetos

Sobrecarga de operadores en C++

Agustín J. González

Patricio Olivares R.

Universidad Técnica Federico Santa María

Propósito de la clase

En C++ podemos definir cómo se comportan operadores como `+`, `=`, `<<` o `[]` cuando se aplican a objetos de nuestras clases.

Esto permite que una clase se use con una sintaxis natural.

```
CVector c = a + b;  
std::cout << c;
```

La sobrecarga de operadores debe hacer que el código sea más legible, no más sorprendente.

Contenidos

1. Motivación
2. Sintaxis general de `operator`
3. Operadores como métodos
4. Operadores como funciones globales
5. Sobrecarga de `+`
6. Sobrecarga de `<<`
7. Operador de asignación
8. Operadores prefijo y postfijo
9. Reglas y buenas prácticas

El problema original

Con tipos fundamentales, esta expresión es natural.

```
int a, b, c;  
c = a + b;
```

Pero una clase definida por nosotros no recibe automáticamente una suma con significado propio.

```
CVector a(3, 1);  
CVector b(1, 2);  
CVector c = a + b; // solo funciona si definimos operator+
```

Qué significa sobrecargar

Sobrecargar un operador significa definir una función con nombre especial.

```
tipo operator signo(parametros);
```

Ejemplo:

```
CVector operator+(const CVector &other) const;
```

El operador sigue siendo `+`, pero el compilador sabe qué función llamar cuando aparecen objetos `CVector`.

No cambia la precedencia

La sobrecarga no permite cambiar reglas sintácticas del lenguaje.

```
a + b * c
```

El `*` se evalúa antes que el `+`, aunque ambos operadores estén sobrecargados.

Tampoco se pueden inventar operadores nuevos.

```
a ** b // no se puede definir como nuevo operador
```

Ejemplo base: CVector

Queremos modelar un vector bidimensional.

```
class CVector {  
public:  
    int x, y;  
  
    CVector() {}  
    CVector(int, int);  
    CVector operator+(const CVector &) const;  
};
```

Para comenzar, los atributos están públicos, como en el ejemplo original.

Luego veremos cómo hacerlo manteniendo encapsulamiento.

Constructor

```
CVector::CVector(int a, int b) {  
    x = a;  
    y = b;  
}
```

El constructor deja el vector en un estado simple:

```
CVector(3, 1) representa (3, 1)
```

Ahora podemos definir qué significa sumar dos vectores.

Sobrecarga de operator+

```
CVector CVector::operator+(const CVector &param) const {  
    CVector temp;  
    temp.x = x + param.x;  
    temp.y = y + param.y;  
    return temp;  
}
```

La suma no modifica los operandos.

Por eso `param` es referencia constante y el método completo termina en `const`.

Uso de operator+

```
CVector a(3, 1);  
CVector b(1, 2);  
CVector c;  
  
c = a + b;
```

La expresión anterior se entiende como:

```
c = a.operator+(b);
```

El objeto izquierdo decide qué implementación de `operator+` se invoca.

Resultado esperado

```
CVector a(3, 1);  
CVector b(1, 2);  
CVector c = a + b;
```

La operación produce:

$$(3 + 1, 1 + 2) = (4, 3)$$

La semántica es intuitiva porque coincide con la idea matemática de suma de vectores.

Operador como método

Un operador método recibe implícitamente el objeto izquierdo.

```
a + b
```

equivale a:

```
a.operator+(b)
```

Por eso un método funciona bien cuando el operando izquierdo es de nuestra clase.

Operador como función global

Un operador global recibe todos sus operandos explícitamente.

```
CVector operator*(double factor, const CVector &v);
```

Esto permite expresiones donde el operando izquierdo no pertenece a nuestra clase.

```
CVector scaled = 3.0 * v;
```

Aquí `3.0` no puede tener un método `operator*` que conozca `CVector`.

Comparación de formas

Expresión	Como método	Como función global
<code>a + b</code>	<code>a.operator+(b)</code>	<code>operator+(a, b)</code>
<code>-a</code>	<code>a.operator-()</code>	<code>operator-(a)</code>
<code>std::cout << a</code>	No se puede modificar <code>ostream</code>	<code>operator<<(std::cout, a)</code>
<code>3.0 * a</code>	No aplica si <code>3.0</code> es <code>double</code>	<code>operator*(3.0, a)</code>

La forma correcta depende de quién aparece al lado izquierdo.

Conexión con friend

Si el operador global necesita leer atributos privados, la clase puede declararlo `friend`.

```
class CVector {  
public:  
    friend CVector operator*(double factor,  
                             const CVector &v);  
  
private:  
    double x, y;  
};
```

La función sigue siendo global, pero recibe permiso para acceder al estado interno.

operator<<

La salida por consola suele implementarse como función global.

```
std::ostream &operator<<(std::ostream &os,  
                        const CVector &v) {  
    os << "(" << v.x << "," << v.y << " )";  
    return os;  
}
```

El retorno por referencia permite encadenar.

```
std::cout << "v = " << v << std::endl;
```


Encadenamiento

La expresión:

```
std::cout << "v = " << v << std::endl;
```

se evalúa como una secuencia de escrituras sobre el mismo flujo.

Cada `operator<<` retorna el flujo recibido.

Por eso la firma correcta es:

```
std::ostream &operator<<(std::ostream &os,  
                        const CVector &v);
```

Operador de asignación

El operador `=` ya existe por defecto para clases.

```
b = a;
```

Por omisión, copia los miembros no estáticos de `a` hacia `b`.

Si todos los miembros son valores simples, esto puede ser suficiente.

Si la clase posee recursos dinámicos, como punteros a memoria del heap, puede ser peligroso.

Asignación y memoria dinámica

En la clase anterior vimos este patrón:

```
class Student {  
private:  
    std::string *courses;  
    int count;  
};
```

La asignación por defecto copiaría la dirección almacenada en `courses`.

Eso genera copia baja.

Por eso se sobrecarga `operator=`.

Forma típica de operator=

```
Student &Student::operator=(const Student &other) {  
    if (this == &other) {  
        return *this;  
    }  
  
    // liberar recurso anterior  
    // copiar recurso de other  
  
    return *this;  
}
```

El retorno `Student &` permite expresiones como:

```
a = b = c;
```

Prefijo y postfijo

Algunos operadores tienen dos formas.

```
++i; // prefijo  
i++; // postfijo
```

En una clase se distinguen por la firma.

```
Counter &operator++(); // prefijo  
Counter operator++(int); // postfijo
```

El `int` del postfijo es un parámetro ficticio usado solo para diferenciar.

Ejemplo conceptual

```
Counter &Counter::operator++() {  
    value++;  
    return *this;  
}  
  
Counter Counter::operator++(int) {  
    Counter old(*this);  
    value++;  
    return old;  
}
```

El prefijo retorna el objeto ya incrementado.

El postfijo retorna una copia del estado anterior.

Operadores que deben ser métodos

Algunos operadores solo pueden sobrecargarse como funciones miembro.

Operador	Uso típico
=	Asignación
[]	Acceso por índice
()	Objeto invocable
->	Acceso indirecto

Esto ocurre porque dependen fuertemente del objeto izquierdo.

Operadores comunes

Tipo de expresión	Operadores frecuentes
Aritmética	<code>+</code> , <code>-</code> , <code>*</code> , <code>/</code> , <code>%</code>
Comparación	<code>==</code> , <code>!=</code> , <code><</code> , <code><=</code> , <code>></code> , <code>>=</code>
Modificación	<code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code>
Entrada/salida	<code><<</code> , <code>>></code>
Acceso	<code>[]</code> , <code>()</code>

No es necesario sobrecargar todo.

Solo se sobrecarga lo que tiene una interpretación clara para la clase.

Reglas del lenguaje

No se puede:

- Crear operadores nuevos.
- Cambiar cantidad de operandos.
- Cambiar precedencia o asociatividad.
- Cambiar el significado de operadores sobre tipos fundamentales solamente.

Ejemplo:

```
int operator+(int a, int b); // no permitido
```

Debe participar al menos un tipo definido por el usuario.

Buenas prácticas

Un operador sobrecargado debe respetar expectativas.

```
a + b
```

debería parecer una operación que no modifica `a` ni `b`.

```
a += b
```

debería modificar `a`.

Si un operador hace algo inesperado, el programa compila, pero se vuelve difícil de leer.

const en operadores

Los operadores que solo observan deben usar `const`.

```
CVector operator+(const CVector &other) const;  
bool operator==(const CVector &other) const;
```

Ventajas:

- Se pueden usar con objetos constantes.
- El compilador verifica que no modifiquen el objeto.
- La intención queda visible en la interfaz.

Eficiencia de parámetros

Para objetos no triviales, evitar copias innecesarias.

```
CVector operator+(const CVector &other) const;
```

Usamos `const CVector &` porque:

- No copiamos el operando derecho.
- No permitimos modificarlo.

El resultado sí se retorna por valor, porque es un objeto nuevo.

Conexión con diseño de clases

La sobrecarga de operadores no es solo sintaxis.

También define la interfaz de una clase.

Decisión	Pregunta
Método o global	¿Quién está al lado izquierdo?
<code>friend</code> o getters	¿Se necesita acceso interno?
Retorno por valor o referencia	¿Se crea un objeto nuevo?
<code>const</code>	¿La operación modifica el objeto?

Ideas clave

- Un operador sobrecargado es una función con nombre `operator` .
- Los operadores como método usan implícitamente el operando izquierdo.
- Las funciones globales permiten casos como `3.0 * v` y `std::cout << v` .
- `friend` puede dar acceso a operadores globales sin hacer públicos los atributos.
- `operator=` debe revisarse si la clase administra recursos.
- Prefijo y postfijo se distinguen por la firma.
- Sobrecargar operadores debe mejorar la legibilidad y respetar expectativas.